Software Architecture Research Group
Faculty for Computer Science
dse@swa.univie.ac.at

Distributed Systems Engineering Work Sheet: Messaging Routing, Adaptation, and Integration

1. Task: Messaging-based System Design

Design a system based on messaging concepts in which order requests containing one or more order items are sent by consumers. The system then sends for each item of the order, a request for a quote on the order item to three different vendors. From all the vendor quotes, the system selects the best quote for each item, once all quotes for each item have arrived. If a vendor does not deliver quote in three minutes, the system continues with the quotes received so far. Once all three best quotes have been gathered, the system creates a combined "validated order" that offers each item with the best received quote and sends it back to the consumer as a reply. Design this system with a figure similar to the ones in the lecture. You can use any pattern introduced in the lecture. If you need to introduce new items or concepts, explain them in one sentence.

2. Task: Studying DSL Code for Message Processing

Study the following code in Apache Camel's DSL (Domain-Specific Language) and Processor interface. What does the code do? Is it related to one or more of the patterns from the lecture?

```
from("direct:start").process(new Processor() {
    public void process(Exchange exchange) {
        Message in = exchange.getIn();
        in.setBody("Hello " + in.getBody(String.class));
    }
}).to("mock:result");
```

3. Task: Studying DSL Code for Message Processing

Study the following code in Apache Camel's DSL (Domain-Specific Language) and Processor interface. What does the code do? Is it related to one or more of the patterns from the lecture?

```
from("direct:start")
    .filter().method(MyBean.class, "isGoldCustomer").to("mock:result").end()
    .to("mock:end");

public static class MyBean {
    public boolean isGoldCustomer(@Header("level") String level) {
        return level.equals("gold");
    }
}
```

4. Task: Channel Adapter vs. Messaging Gateway

Discuss the main differences of the Channel Adapter and Messaging Gateway patterns.

5. Task: Normalizer

Suppose there are three stock quote clients sending SOAP, Plain Old XML (POX), or JSON Stock Quote Requests. Consider you can use XPath expressions for inspecting message content on all three types of requests and XSLT for transforming the content (some solutions for JSON exist). Can you use these technologies to build a normalizer for these requests? If yes: How would you design it and which components need to be implemented? If no: why not?

6. Task: Splitters and Aggregators in Spring Integration

Study the documentation of Splitter and Aggregator in Spring Integration, e.g. at:

- http://docs.spring.io/spring-integration/docs/2.0.0.RC1/reference/html/splitter.html
- http://docs.spring.io/spring-integration/docs/2.0.0.RC1/reference/html/aggregator.html

a. How do the two pattern realizations use the correlation ID?
b. Why does Aggregator use a Message Store?
c. What is an Aggregator release strategy? Give an example what it is used for?